

REPUBLICQUE DU CAMEROUN

Paix – Travail – Patrie

MINISTERE DE
L'ENSEIGNEMENT
SUPERIEUR

UNIVERSITE DE YAOUNDE 1

SAINT JEAN INGENIEUR



REPUBLIC OF CAMEROON

Peace– Work– Fatherland

MINISTRY OF HIGHER
EDUCATION

UNIVERSITY OF YAOUNDE 1

SAINT JEAN INGENIEUR



CAHIER DES CHARGES :

MédiSecours+

Application web de premiers soins, de géolocalisation des centres de santé et de mise en relation avec des médecins

Rédigé et Présenté par
**NGABA SIKOMBE ROSTAND
ALBERT**
Etudiant en Licence 2

Académique de :
MR DJOUFACK VINCENT

ANNEE ACADEMIQUE :
2025 – 2026

Sommaire

Sommaire.....	2
1. Présentation générale du projet.....	3
1.1 Contexte	3
1.2 Problématique.....	3
1.3 Objectifs du projet	3
1.4 Public cible.....	3
2. Périmètre du projet.....	4
2.1 Fonctionnalités incluses dans la version 1 (MVP)	4
2.2 Hors périmètre (évolutions futures envisageables).....	4
3. Acteurs du système	4
4. Description fonctionnelle détaillée	5
4.1 Module Authentification et gestion des comptes.....	5
4.2 Module Catalogue de premiers soins	5
4.3 Module Géolocalisation des centres de santé	5
4.4 Module Mise en relation avec un médecin	5
4.5 Module Administration (back-office)	5
4.6 Module Notifications.....	6
5. Exigences non fonctionnelles	6
6. Architecture technique	7
6.1 Architecture générale	7
6.2 Frontend — React.js	7
6.3 Backend — Symfony.....	7
6.4 Base de données.....	7
6.5 Services et API tiers.....	7
6.6 Sécurité technique	8
7. Modélisation des données	8
8. Cas d'utilisation principaux	9
9. Parcours utilisateurs clés.....	9
9.1 Parcours « Patient en situation d'urgence »	9
9.2 Parcours « Médecin »	9
10. Méthodologie de gestion de projet.....	10
11. Planning prévisionnel	10
12. Livrables attendus	11
13. Contraintes du projet	11
14. Risques et mesures d'atténuation.....	12
15. Critères de validation et de réception.....	12
16. Annexe — Glossaire.....	13

1. Présentation générale du projet

1.1 Contexte

Face à un problème de santé, qu'il s'agisse d'un malaise, d'une blessure ou d'une maladie soudaine, les personnes concernées manquent souvent d'un accès rapide et fiable à deux types d'informations essentielles : les gestes de premiers soins adaptés à la situation, et les structures de santé disponibles à proximité. Cette perte de temps, ainsi que la difficulté à entrer rapidement en contact avec un professionnel de santé, peuvent aggraver l'état du patient ou retarder une prise en charge appropriée.

Le projet vise à répondre à ce besoin par une application web centralisant ces trois services : information médicale de premiers secours, géolocalisation des centres de santé, et mise en relation directe avec un médecin.

1.2 Problématique

Comment proposer aux utilisateurs un outil web unique, simple et accessible, permettant : (1) d'identifier rapidement les gestes de premiers soins adaptés à un symptôme ou une maladie donnée.

(2) de localiser le centre de santé le plus proche en cas de besoin.

(3) de contacter directement un médecin pour obtenir un avis ou organiser une prise en charge rapide ?

1.3 Objectifs du projet

Objectif général : concevoir et développer une application web d'orientation et d'aide médicale de proximité, basée sur les technologies React.js (frontend) et Symfony (backend).

Objectifs spécifiques :

- Mettre à disposition un catalogue fiable et structuré de premiers soins, organisé par maladie ou symptôme.
- Géolocaliser l'utilisateur et afficher sur une carte interactive les centres de santé les plus proches.
- Permettre une mise en relation directe (messaging, demande de rappel ou de rendez-vous) entre un patient et un médecin.
- Garantir une expérience utilisateur simple, rapide et accessible sur ordinateur, tablette et mobile.
- Sécuriser les données personnelles et médicales des utilisateurs, conformément aux bonnes pratiques de protection des données.

1.4 Public cible

- Particuliers / patients souhaitant une réponse rapide à un problème de santé.
- Médecins et professionnels de santé souhaitant être visibles et contactés via la plateforme.
- Centres de santé, cliniques et pharmacies souhaitant être référencés sur la carte.
- Administrateurs en charge de la gestion du contenu et de la modération de la plateforme.

2. Périmètre du projet

2.1 Fonctionnalités incluses dans la version 1 (MVP)

- Catalogue de premiers soins par maladie / symptôme, avec recherche et filtres.
- Géolocalisation et cartographie des centres de santé les plus proches.
- Mise en relation directe avec un médecin (messagerie instantanée, demande de rappel ou de rendez-vous).
- Gestion des comptes utilisateurs (patient, médecin, administrateur).
- Back-office d'administration du contenu et des utilisateurs.
- Notifications (messages, validation de compte, confirmation de rendez-vous).

2.2 Hors périmètre (évolutions futures envisageables)

- Téléconsultation vidéo complète intégrée avec paiement en ligne.
- Prescription électronique et ordonnances numériques.
- Application mobile native iOS / Android (la V1 est une application web responsive).
- Diagnostic automatique assisté par intelligence artificielle.
- Intégration avec les dossiers médicaux électroniques des hôpitaux.

3. Acteurs du système

Le tableau ci-dessous présente les différents acteurs amenés à interagir avec la plateforme.

Acteur	Description	Droits principaux
Visiteur	Internaute non connecté	Consulter le catalogue de premiers soins ; visualiser les centres de santé sur la carte
Patient	Utilisateur connecté cherchant de l'aide	Tout ce que fait le visiteur, + gérer son profil, contacter un médecin, consulter son historique, laisser un avis
Médecin	Professionnel de santé référencé	Gérer son profil professionnel et ses disponibilités, recevoir et répondre aux demandes des patients
Administrateur	Gestionnaire de la plateforme	Gérer le catalogue, les centres de santé, valider les comptes médecins, modérer les avis, consulter les statistiques

4. Description fonctionnelle détaillée

4.1 Module Authentification et gestion des comptes

- Inscription patient et médecin par email / mot de passe, avec vérification de l'adresse email.
- Connexion, déconnexion, et réinitialisation du mot de passe en cas d'oubli.
- Gestion du profil utilisateur (informations personnelles, photo, localisation).
- Pour les médecins : validation manuelle du compte par un administrateur (vérification du diplôme / numéro d'ordre) avant activation.

4.2 Module Catalogue de premiers soins

- Liste des maladies et symptômes classés par catégorie (brûlures, fractures, malaises, allergies, intoxications, plaies, etc.).
- Fiche détaillée par maladie : description, signes et symptômes, gestes de premiers soins à effectuer, gestes à éviter, niveau d'urgence (faible / modéré / critique), recommandation de consulter ou non un professionnel.
- Moteur de recherche et filtres (par catégorie, mot-clé, niveau d'urgence).
- Contenu rédigé et validé par l'administrateur ou des professionnels de santé partenaires, afin de garantir sa fiabilité.

4.3 Module Géolocalisation des centres de santé

- Détection automatique de la position de l'utilisateur via l'API de géolocalisation du navigateur, avec possibilité de saisie manuelle de l'adresse.
- Affichage sur une carte interactive des centres de santé les plus proches (hôpitaux, cliniques, cabinets, pharmacies).
- Fiche centre de santé : nom, adresse, téléphone, horaires d'ouverture, spécialités disponibles, distance et itinéraire.
- Tri par distance et filtrage par type d'établissement ou par spécialité.
- Calcul d'itinéraire vers le centre choisi (intégration d'une API cartographique).

4.4 Module Mise en relation avec un médecin

- Recherche de médecin par spécialité, disponibilité ou proximité géographique.
- Fiche médecin : nom, spécialité, expérience, avis et notes, disponibilités.
- Contact direct via une messagerie instantanée intégrée à la plateforme.
- Demande de rappel téléphonique ou prise de rendez-vous en ligne.
- Historique des échanges et des demandes consultable par le patient et le médecin.
- Système de notation et d'avis laissé par le patient après l'échange.

4.5 Module Administration (back-office)

- Gestion complète (création, modification, suppression) du catalogue : maladies, catégories, fiches de premiers soins.
- Gestion complète des centres de santé référencés.
- Validation et gestion des comptes médecins.

- Modération des avis et des messages signalés.
- Tableau de bord de statistiques d'usage (utilisateurs inscrits, mises en relation, recherches fréquentes...).

4.6 Module Notifications

- Notifications en temps réel : nouveau message, rendez-vous confirmé, compte médecin validé.
- Notifications par email pour les actions importantes (inscription, réinitialisation de mot de passe, confirmation de rendez-vous).

5. Exigences non fonctionnelles

Exigence	Description
Performance	Temps de chargement des pages inférieur à 2-3 secondes ; recherche dans le catalogue quasi instantanée.
Sécurité	Mots de passe hachés (bcrypt / argon2), connexions en HTTPS, gestion des rôles (RBAC), respect des principes de protection des données personnelles et de santé.
Disponibilité	Application accessible 24h/24, hébergement avec sauvegardes régulières de la base de données.
Ergonomie / Accessibilité	Interface simple et intuitive, design responsive (mobile, tablette, ordinateur), bonnes pratiques d'accessibilité.
Scalabilité	Architecture découplée front / back permettant l'ajout futur de fonctionnalités (téléconsultation, application mobile) sans refonte complète.
Maintenabilité	Code structuré, documenté, respect des conventions Symfony et React, séparation claire des responsabilités.
Compatibilité	Fonctionnement sur les navigateurs principaux : Chrome, Firefox, Edge, Safari.

6. Architecture technique

6.1 Architecture générale

L'application repose sur une architecture découplée de type SPA (Single Page Application) côté client, associée à une API REST côté serveur :

- Frontend : application React.js (SPA) consommant les données via des appels API.
- Backend : API développée avec Symfony (PHP), exposant des endpoints REST, gérant la logique métier, l'authentification et l'accès aux données.
- Base de données : système de gestion de base de données relationnel (MySQL ou PostgreSQL), accédé via l'ORM Doctrine.
- Communication : échanges en JSON entre le frontend et le backend, sécurisés par HTTPS.

Schéma logique : Navigateur (React.js) $\rightleftharpoons$ API REST (Symfony) $\rightleftharpoons$ Base de données (MySQL/PostgreSQL), avec appels à des services externes (cartographie, emails, notifications temps réel).

6.2 Frontend — React.js

- React.js avec React Router pour la navigation entre les pages.
- Gestion d'état via Context API, ou Redux / Zustand si la complexité le justifie.
- Axios pour la communication avec l'API REST.
- Bibliothèque de cartographie : Leaflet (OpenStreetMap) ou Google Maps API.
- Bibliothèque de composants UI (ex. Material UI ou Tailwind CSS) pour un rendu responsive et cohérent.
- Gestion et validation des formulaires (ex. React Hook Form, Formik + Yup).

6.3 Backend — Symfony

- Symfony (version récente), avec API Platform pour générer rapidement une API REST documentée (OpenAPI / Swagger).
- Doctrine ORM pour la gestion de la base de données.
- LexikJWTAuthenticationBundle pour l'authentification par jeton JWT.
- Symfony Mailer pour l'envoi des emails (vérification de compte, notifications).
- Mercure (composant Symfony) ou WebSocket pour la messagerie et les notifications en temps réel.
- Voters Symfony pour la gestion fine des autorisations par rôle.

6.4 Base de données

Système de gestion de base de données relationnel : MySQL ou PostgreSQL. Le modèle de données détaillé est présenté en section 7.

6.5 Services et API tiers

- API de cartographie et géolocalisation : Google Maps API ou OpenStreetMap / Leaflet avec Nominatim.
- Service d'envoi d'emails (SMTP, ou outil de test type Mailtrap en développement).
- Solution de messagerie temps réel : Mercure, Socket.io ou service tiers équivalent.

- (Évolution possible) API d'appel ou de visioconférence : WebRTC, Twilio ou Jitsi.

6.6 Sécurité technique

- HTTPS obligatoire sur l'ensemble de l'application.
- Authentification par JWT avec gestion de jetons de renouvellement (refresh tokens).
- Hachage des mots de passe via les mécanismes de sécurité de Symfony (bcrypt / argon2i).
- Gestion des rôles : ROLE_PATIENT, ROLE_MEDECIN, ROLE_ADMIN.
- Protection contre les failles courantes : CSRF, XSS, injection SQL (limitée par l'usage de l'ORM).
- Respect des principes de protection des données personnelles : consentement, minimisation des données, droit à l'oubli.

7. Modélisation des données

Le tableau ci-dessous présente les principales entités envisagées pour le modèle de données. Cette liste est une base de travail à affiner lors de la phase de conception détaillée (diagramme de classes / modèle relationnel).

Entité	Description	Attributs principaux
Utilisateur	Compte de base commun	id, nom, prénom, email, mot de passe, rôle, téléphone, date de création
Patient	Profil patient (lié à Utilisateur)	adresse, latitude, longitude
Médecin	Profil médecin (lié à Utilisateur)	spécialité, numéro d'ordre, statut de validation, disponibilités
CentreDeSante	Établissement de santé référencé	nom, type, adresse, latitude, longitude, téléphone, horaires
Categorie	Catégorie de maladies	id, nom
Maladie	Maladie ou symptôme du catalogue	id, nom, catégorie, description, niveau d'urgence
PremierSoin	Geste de premier soin associé	id, maladie (référence), étapes à suivre, précautions
Consultation	Échange entre patient et médecin	id, patient (référence), médecin (référence), statut, date de création
Message	Message au sein d'une consultation	id, consultation (référence), expéditeur, contenu, date d'envoi
Avis	Avis laissé par un patient	id, médecin (référence), patient (référence), note, commentaire

8. Cas d'utilisation principaux

Acteur	Cas d'utilisation	Description
Visiteur	Consulter le catalogue	Parcourir et rechercher les fiches de premiers soins sans créer de compte.
Visiteur	Localiser un centre de santé	Visualiser sur une carte les centres de santé proches de sa position.
Patient	Créer un compte / se connecter	S'inscrire ou se connecter pour accéder aux fonctionnalités avancées.
Patient	Contacteur un médecin	Rechercher un médecin et lui envoyer un message ou une demande de rappel.
Patient	Laisser un avis	Noter et commenter un médecin après un échange.
Médecin	Gérer son profil	Mettre à jour ses informations, sa spécialité et ses disponibilités.
Médecin	Répondre aux patients	Consulter et répondre aux demandes reçues via la messagerie.
Administrateur	Gérer le catalogue	Créer, modifier ou supprimer des maladies et des fiches de premiers soins.
Administrateur	Valider un médecin	Vérifier les informations d'un médecin et activer son compte.
Administrateur	Modérer la plateforme	Gérer les avis et messages signalés par les utilisateurs.

9. Parcours utilisateurs clés

9.1 Parcours « Patient en situation d'urgence »

1. Le patient arrive sur l'application et recherche son symptôme dans le catalogue.
2. Il consulte la fiche de premiers soins correspondante et applique les gestes recommandés.
3. Si la fiche recommande une consultation, il active sa géolocalisation pour voir les centres de santé proches.
4. Il choisit un centre ou contacte directement un médecin disponible via la messagerie.

9.2 Parcours « Médecin »

1. Le médecin crée son compte et complète son profil professionnel.
2. Son compte est vérifié et validé par un administrateur.
3. Il indique ses disponibilités et reçoit les demandes des patients.
4. Il échange avec les patients via la messagerie et organise un suivi si nécessaire.

10. Méthodologie de gestion de projet

Il est recommandé de conduire ce projet selon une méthodologie agile (type Scrum), avec des sprints d'une à deux semaines, permettant de livrer progressivement les modules fonctionnels et d'ajuster les priorités.

Outils suggérés :

- Gestion du code source : Git / GitHub ou GitLab, avec une stratégie de branches (ex. Gitflow).
- Suivi des tâches : Trello, Jira ou Notion.
- Maquettage : Figma.
- Documentation de l'API : Swagger / OpenAPI (généré via API Platform).

11. Planning prévisionnel

Phase	Description	Durée estimée
Analyse & conception	Cahier des charges, maquettes UI/UX, modélisation de la base de données	2 semaines
Mise en place de l'architecture	Initialisation du projet Symfony (API), du projet React, configuration de la base de données	1 semaine
Module Authentification	Inscription, connexion, gestion des rôles et des profils	1 à 2 semaines
Module Catalogue de premiers soins	CRUD maladies/fiches, recherche et filtres	2 semaines
Module Géolocalisation	Intégration carte, géolocalisation, fiches centres de santé	2 semaines
Module Contact médecin	Messagerie, demandes de rappel/rendez-vous, avis	2 à 3 semaines
Back-office administrateur	Gestion du contenu, validation des médecins, statistiques	1 à 2 semaines
Tests & optimisation	Tests fonctionnels, corrections, optimisation des performances	2 semaines
Déploiement & recette	Mise en production, recette finale avec le porteur de projet	1 semaine

Durée totale estimée : environ 14 à 16 semaines (3 à 4 mois), à ajuster selon les ressources disponibles.

12. Livrables attendus

- Le présent cahier des charges, validé par le porteur de projet.
- Les maquettes UI/UX (Figma ou équivalent).
- Le code source du frontend (React) et du backend (Symfony), versionné sur un dépôt Git.
- La documentation technique et la documentation de l'API (Swagger).
- La base de données et les scripts de migration (Doctrine Migrations).
- L'application déployée sur un environnement de recette puis de production.
- Un manuel d'utilisation à destination des utilisateurs finaux.

13. Contraintes du projet

- Techniques : utilisation imposée de React.js pour le frontend et de Symfony pour le backend.
- Temporelles : respect du planning prévisionnel défini en section 11.
- Réglementaires : prise en compte de la sensibilité des données de santé et des principes de protection des données personnelles.
- Budgétaires : coûts d'hébergement et d'éventuelles API tierces payantes (ex. Google Maps au-delà d'un certain quota), avec possibilité d'alternatives gratuites (OpenStreetMap).

14. Risques et mesures d'atténuation

Risque	Impact	Mesure d'atténuation
Imprécision de la géolocalisation	Centres de santé erronés ou mal classés	Permettre la saisie manuelle de l'adresse en complément de la géolocalisation automatique
Fiabilité du contenu médical	Risque pour la santé des utilisateurs	Contenu validé par des professionnels de santé ; mention claire que l'application ne remplace pas un avis médical
Indisponibilité des médecins	Mauvaise expérience utilisateur	Affichage des disponibilités en temps réel et présence de plusieurs médecins par spécialité
Sécurité des données médicales	Fuite de données sensibles	Chiffrement, HTTPS, contrôle d'accès strict basé sur les rôles
Dépassement du planning	Retard de livraison du projet	Approche agile avec priorisation des fonctionnalités essentielles (MVP en premier)

15. Critères de validation et de réception

- L'ensemble des fonctionnalités du périmètre défini (section 2.1) est implémenté et testé.
- L'application est responsive et fonctionne correctement sur les principaux navigateurs.
- Les temps de réponse respectent les exigences de performance définies en section 5.
- Les contrôles de sécurité de base (authentification, gestion des rôles) sont vérifiés.
- La documentation technique est livrée et à jour.

16. Annexe — Glossaire

Terme	Définition
API	Interface de programmation permettant à deux applications de communiquer (ici, entre React et Symfony).
SPA	Single Page Application : application web qui se charge une seule fois et met à jour son contenu dynamiquement.
JWT	JSON Web Token : jeton utilisé pour authentifier les utilisateurs auprès de l'API.
ORM	Object-Relational Mapping : technique permettant de manipuler une base de données via des objets (Doctrine pour Symfony).
RBAC	Role-Based Access Control : gestion des droits d'accès basée sur les rôles des utilisateurs.
MVP	Minimum Viable Product : version minimale d'un produit regroupant les fonctionnalités essentielles.

17. Diagramme de Cas d'Utilisation

Le diagramme ci-dessous présente les cas d'utilisation principaux de l'application MediSecours+, en identifiant les quatre acteurs (Visiteur, Patient, Medecin, Administrateur) et leurs interactions avec les fonctionnalites du systeme. Les relations <<include>> et <<extend>> decrivent les dependances entre cas d'utilisation.

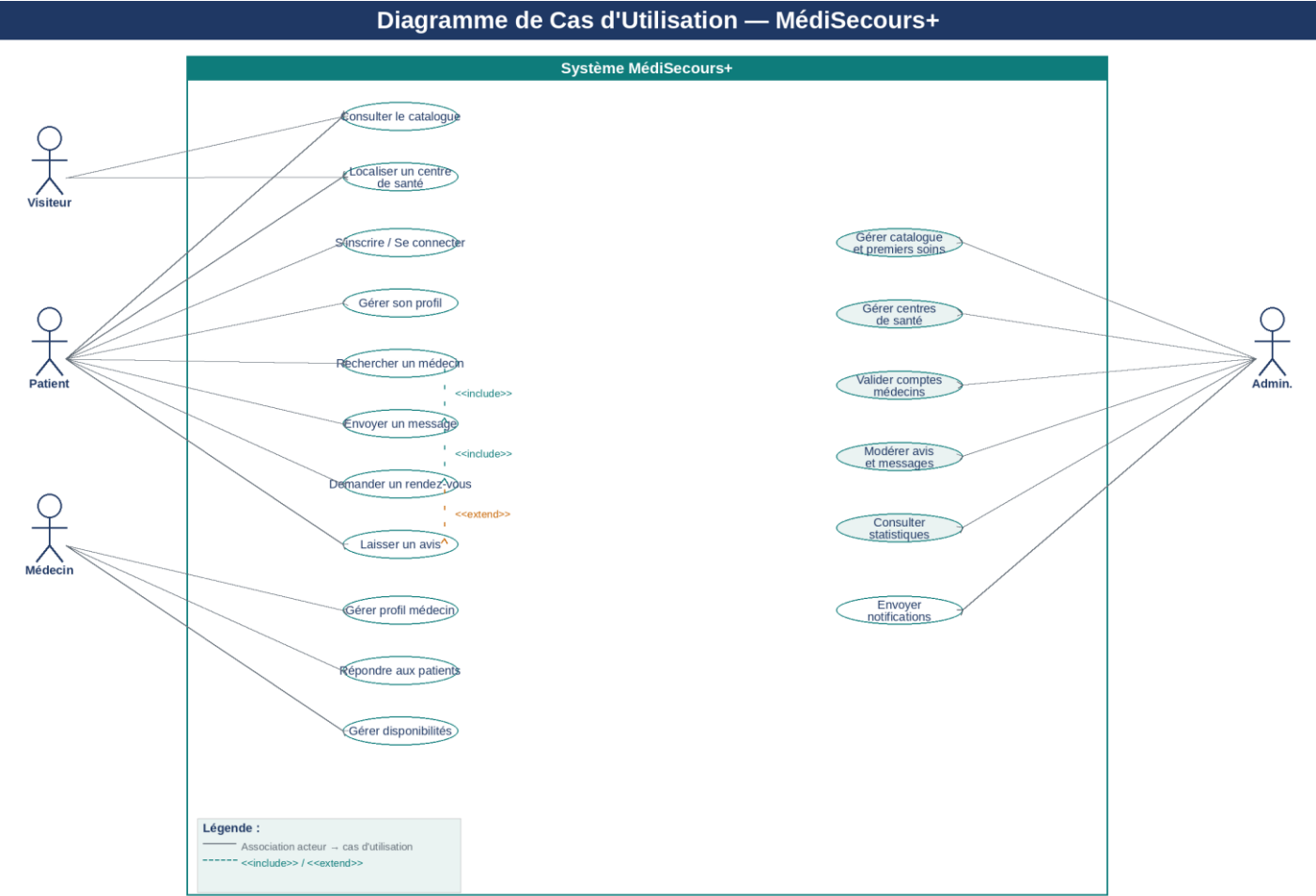


Figure 1 — Diagramme de cas d'utilisation — MediSecours+

18. Diagramme de Classes

Le diagramme de classes décrit la structure statique du système : les entités métier, leurs attributs, leurs méthodes et les relations entre elles (héritage, association, composition). Ce modèle constitue la base du schéma de base de données Doctrine/Symfony.

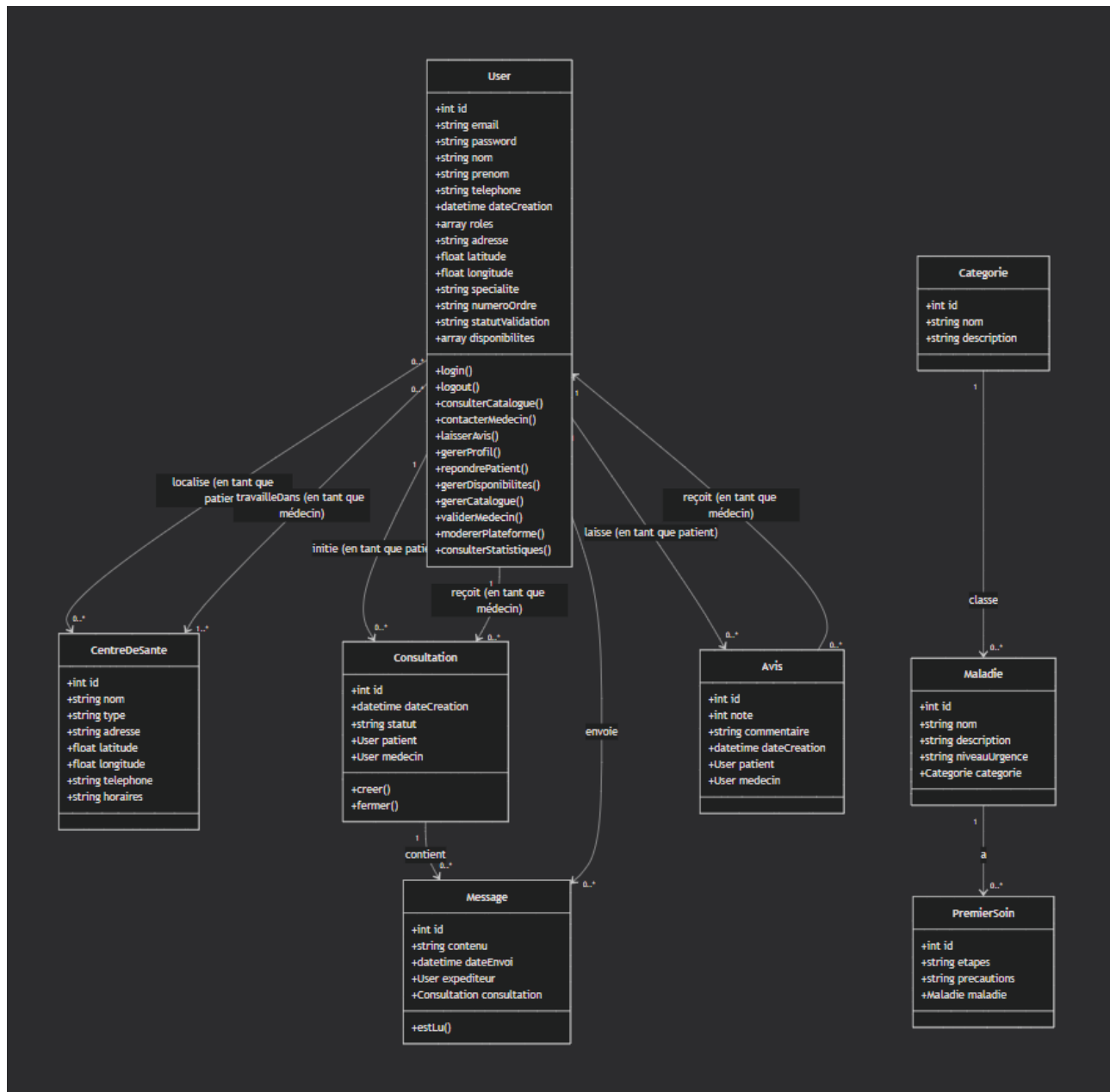


Figure 2 — Diagramme de classes — MediSecours+

19. Diagrammes de Sequence

Les diagrammes de sequence illustrent les echanges dynamiques entre les acteurs et les composants du systeme (navigateur React, API Symfony, base de donnees) pour les trois scenarios principaux de l'application.

19.1 Consultation du catalogue de premiers soins

Ce scenario decrit le parcours complet d'un utilisateur (visiteur ou patient) depuis la saisie d'un mot-cle jusqu'a l'affichage de la fiche detaillee de premiers soins, incluant les appels a l'API REST Symfony et les requetes sur la base de donnees.

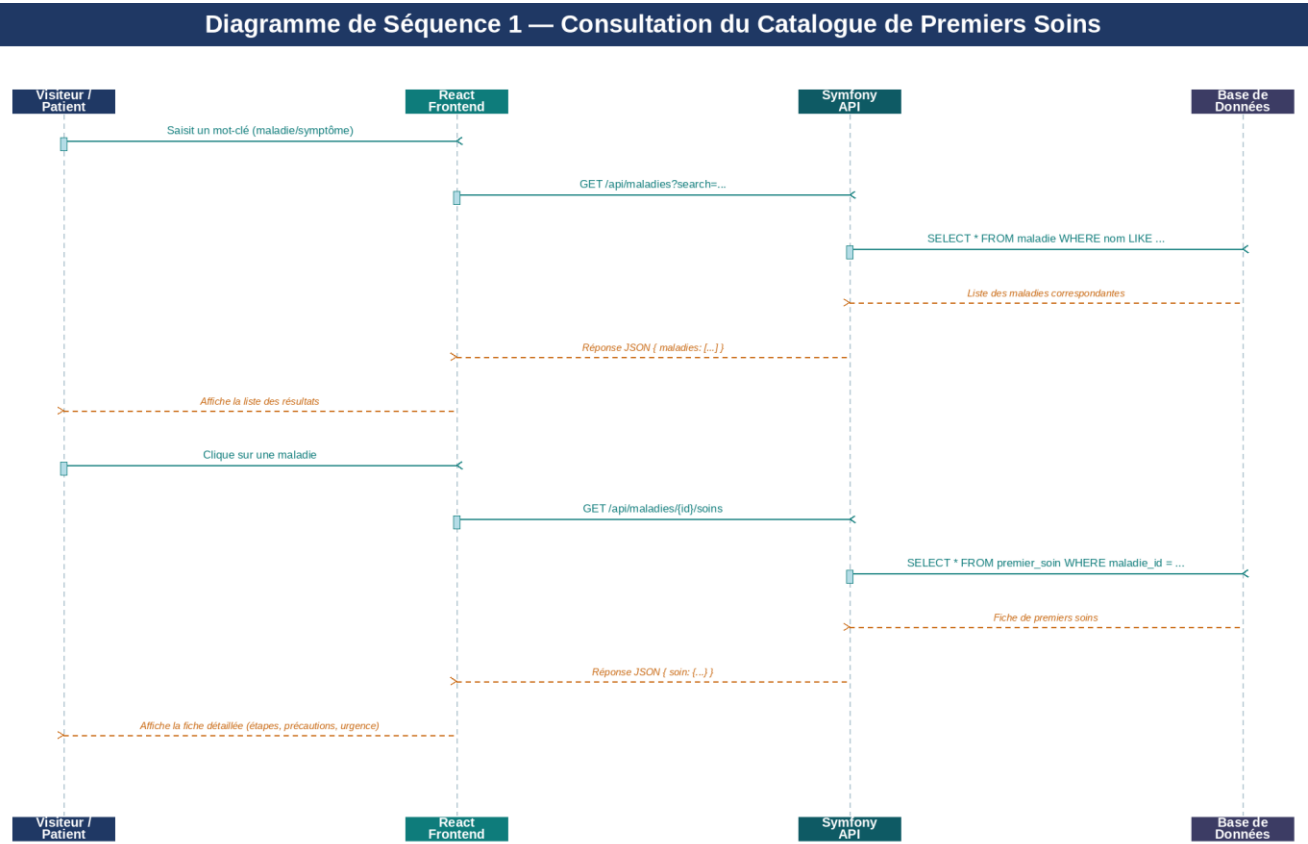


Figure 3 — Sequence : Consultation du catalogue de premiers soins

19.2 Geolocalisation des centres de sante

Ce diagramme montre comment le systeme exploite la geolocalisation du navigateur pour interroger l'API et afficher sur la carte interactive Leaflet les centres de sante les plus proches de la position de l'utilisateur.

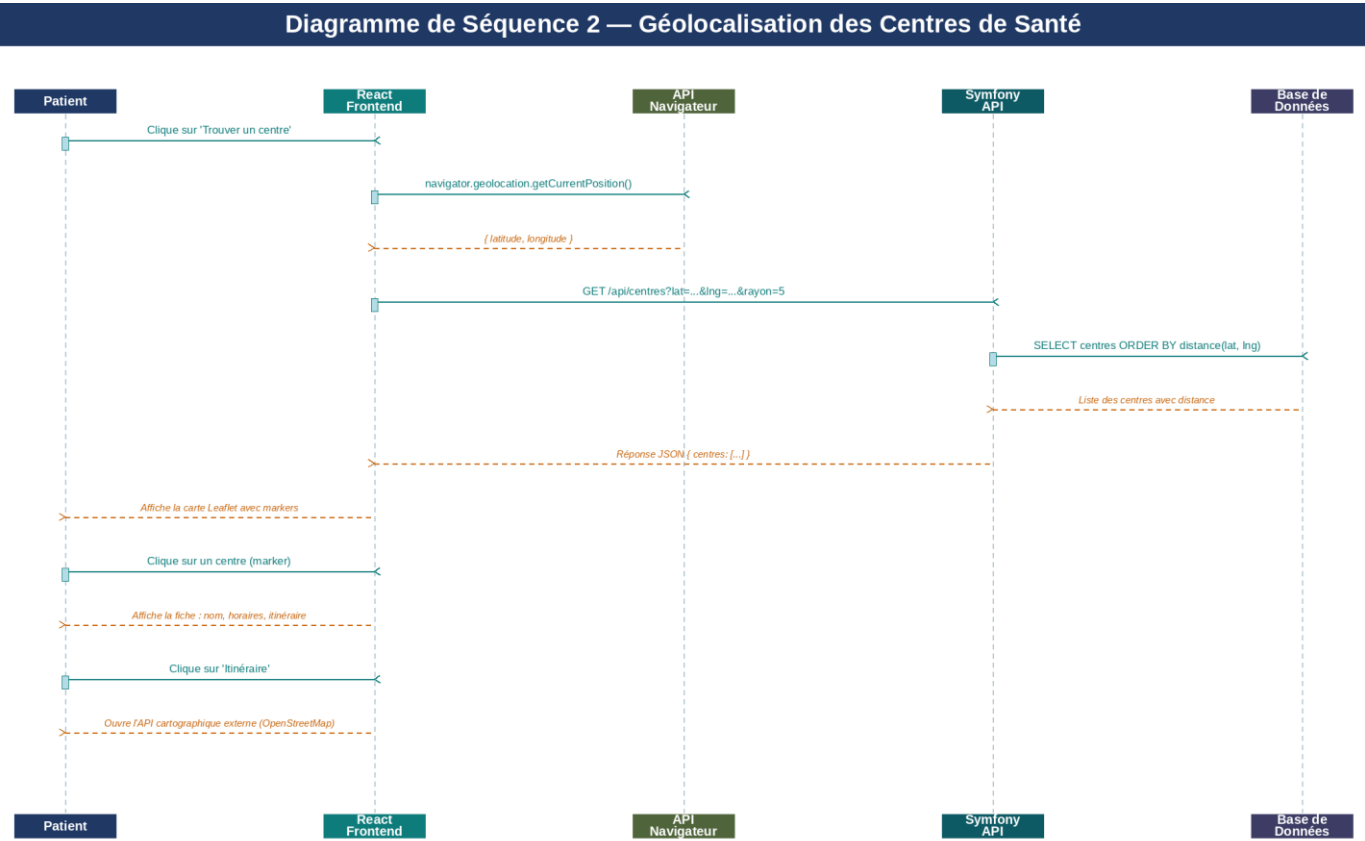


Figure 4 — Sequence : Geolocalisation des centres de sante

19.3 Mise en relation Patient / Medecin

Ce scenario couvre la recherche d'un medecin par le patient, l'envoi d'un premier message, la notification en temps reel au medecin (via Mercure ou WebSocket), la reponse du medecin et la notification de retour au patient.

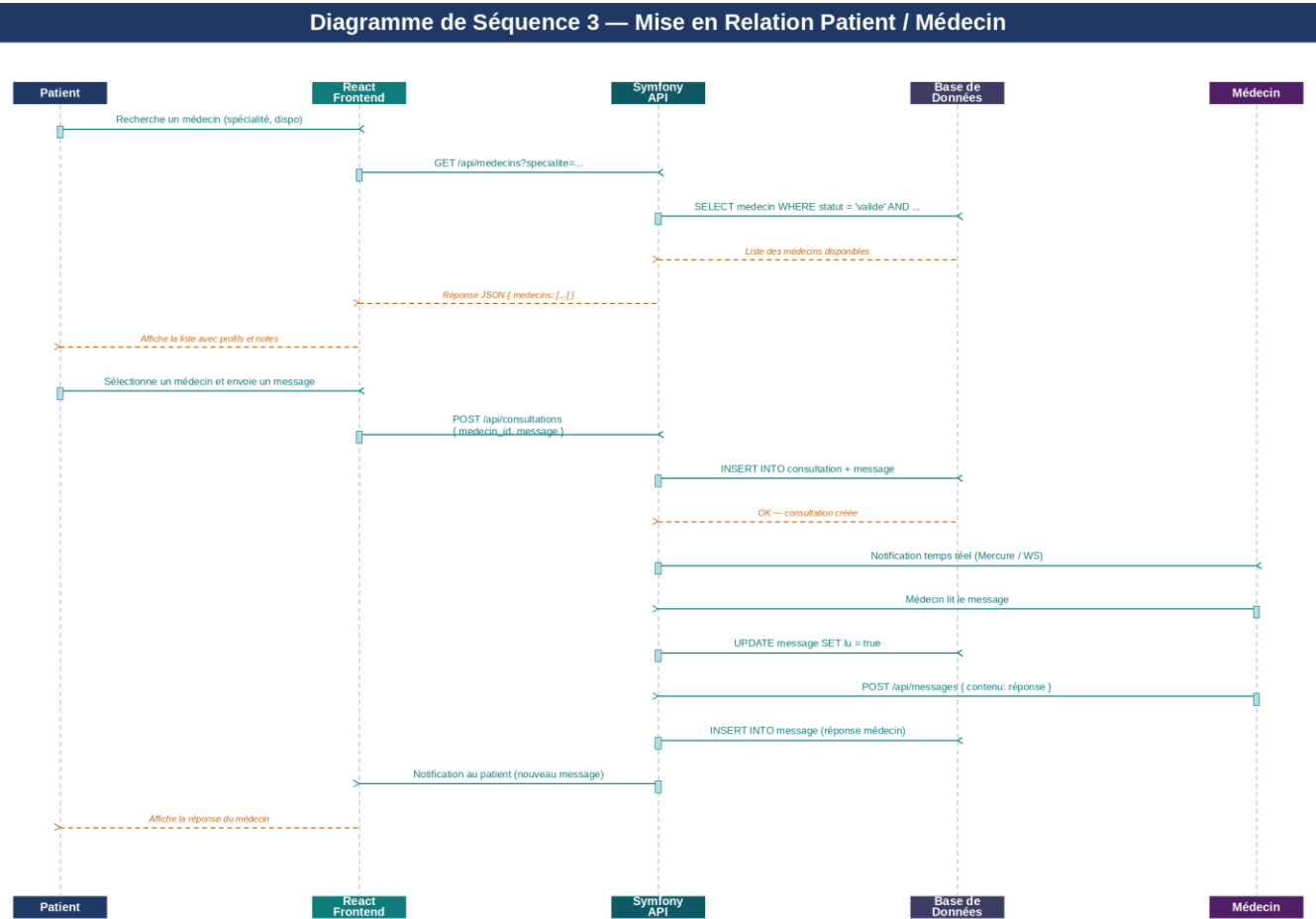


Figure 5 — Sequence : Mise en relation patient / medecin